UNIVERSIDAD D SEVILLA

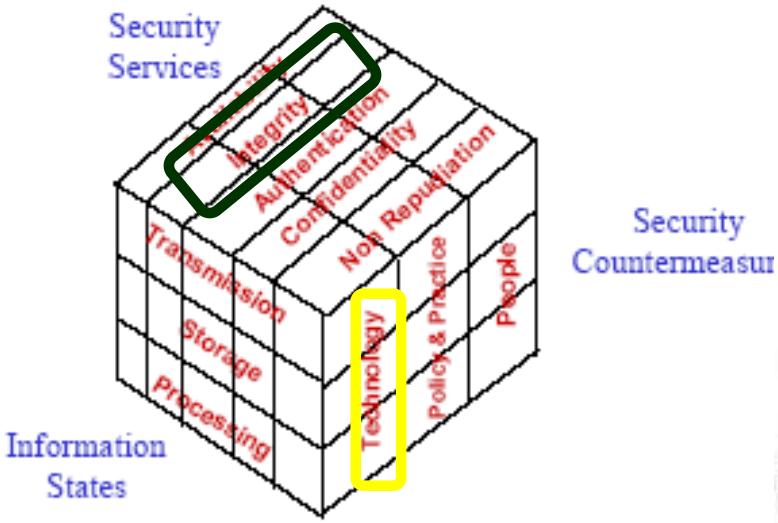
Tema 2. Integridad de la Información

Dr. Rafael Ceballos Guerrero
Grupo de Investigación **Quivir**,
Tecnologías Inteligentes y de Seguridad de los Sistemas de Información
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Sevilla





- **Implementacion Tecnológica de Requisitos de SI**
  - **Integridad**





Implementación de requisitos de la SI

- Objetivos en SI
  - ***Prevenir y detectar*** **ataques (protección/mantenimiento y monitorización en tiempo real)**
  - ***Responder*** de una forma adecuada y con prontitud ante un ***incidente***
- **Tecnológicamente *la seguridad está ya aquí***
  - En las páginas Web y Servicios Web
  - En los Sistemas Operativos.
  - En los lenguajes de programación: Java, C#, Phyton, Ruby..
  - En los frameworks Web: Apache Struts, Django,…
  - En los gestores de contenido/CMS: Joomla, Drupal, …
  - En ***herramientas de seguridad ,*** Tripwire, Eraser, Antivirus, firewalls, ..

Implementación de la Integridad

- Solución técnica a la Integridad de datos: ***Digestión, resumen, huella digital, hash seguro o Compendio con funciones hash***



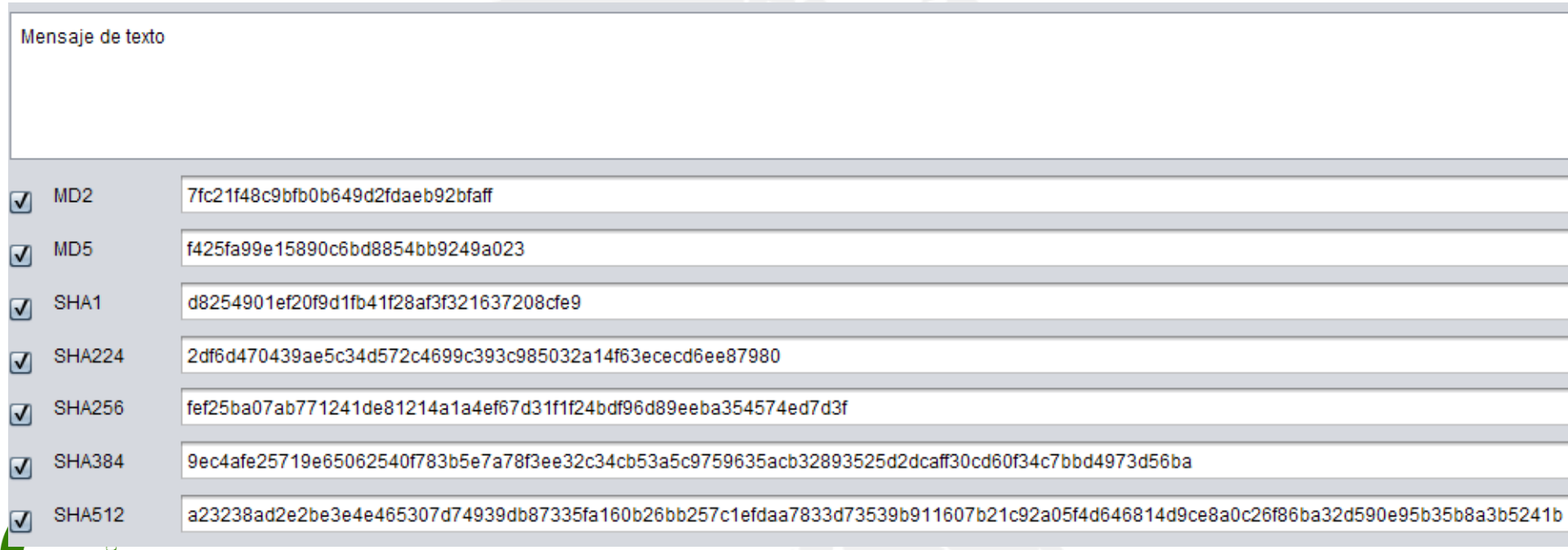
ac8e42e26b3e92eba3922b32744daed008b0a4d1

Implementación de la Integridad

- Un **secure hash** es una ***hash function***;
  - **Input: un bloque de datos abritarios** (***message***)
  - **Output: Una cadena de bit de tamaño prefijado**, el ***hash value*** **(***message digest o simply digest ,fingerprint, checksum***)**, tal que cualquier cambio (accidental o intencionado) de los datos (**con muy elevada probabilidad)** cambiará el ***hash value***.
  - **Propiedades del secure hash:**
    - Es fácil calcular computacionalmente el ***hash value***.
    - Es una función de solo sentido. Es imposible generar el ***message*** de un ***hash value*** dado.
  - **Propiedades ideales del secure hash:**
    - Es imposible modificar un ***message*** sin cambiar el *hash value.*
    - Es imposible encontrar dos ***message*** diferentes con el mismo ***hash value***.

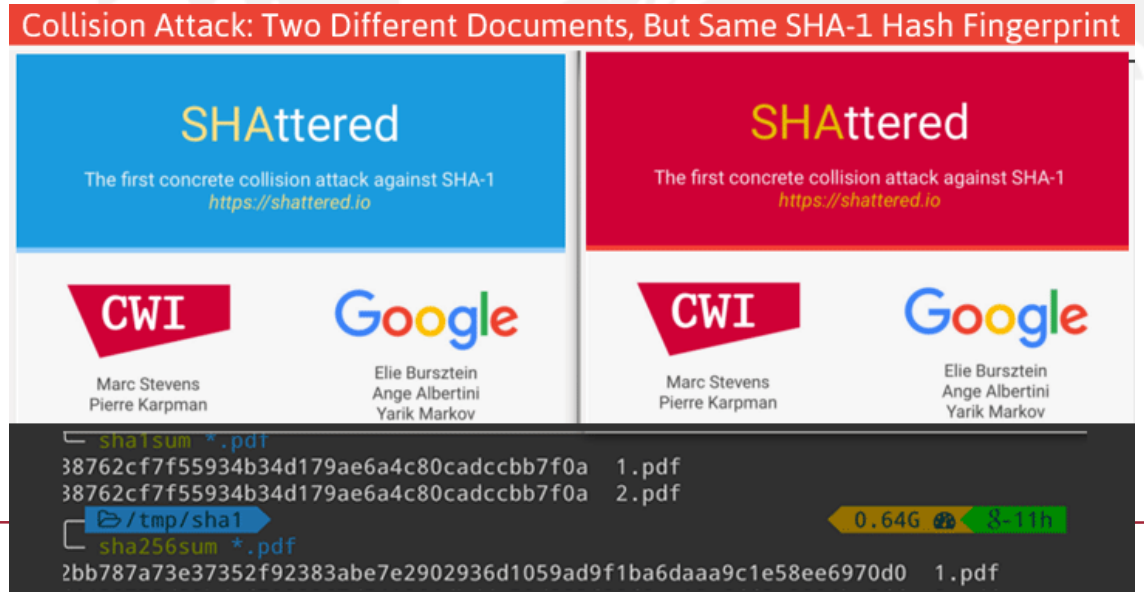
Implementación de la Integridad

- Algoritmos:
  - MD5. Resumen con longitud fija de 128bits. No es segura
  - SHA-1. Bloque de 160bits. Función de compresión más compleja, es más lento, el número de pasos son de 80 (64 en MD5). Se han encontrado colisiones, por tanto, actualmente no es segura



Implementación de la Integridad

- El cifrado SHA-1 ya no es seguro: Google lo ha roto después de 22 años
  - Grupo de investigadores de Google y del CWI de Amsterdam han logrado con éxito un **ataque de colisión** en SHA-1
  - Problema: **han conseguido modificar un mensaje, y el resumen obtenido para el original y el modificado son iguales.**
  - Lamentablemente SHA-1 se sigue utilizando a día de hoy



Implementación de la Integridad

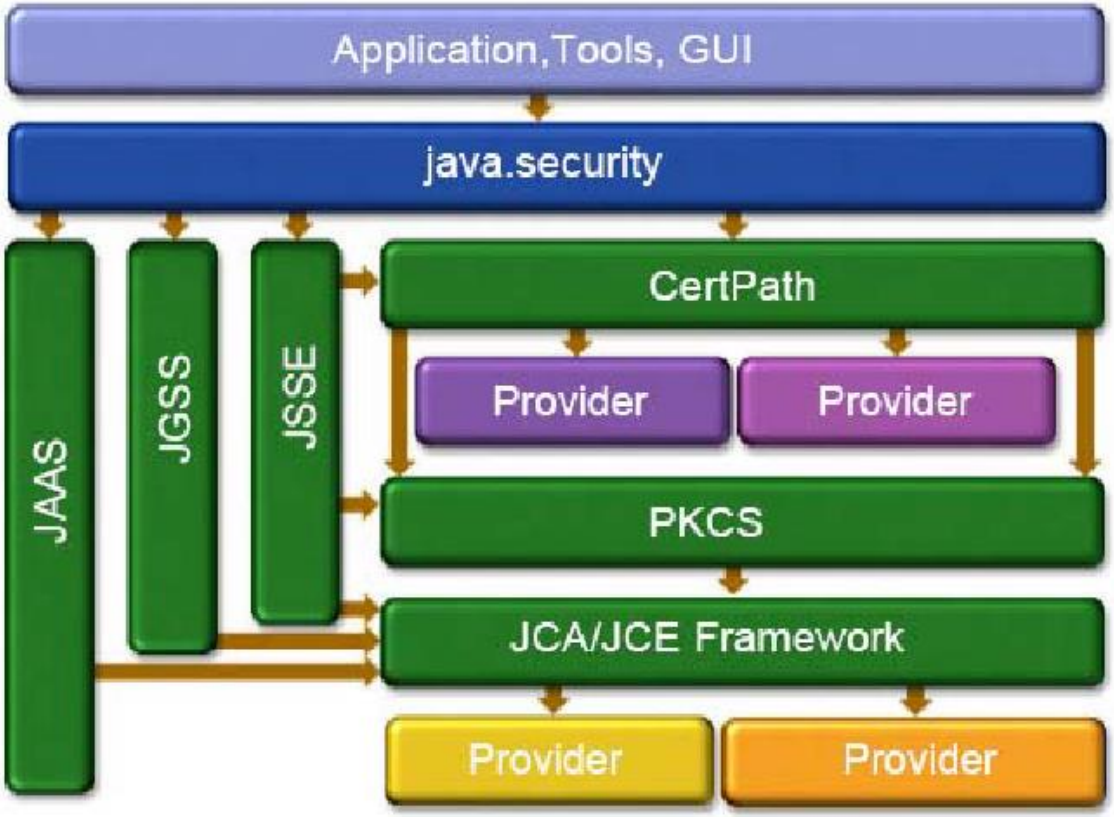
- Algoritimos:
  - SHA-224, SHA-256, SHA-384 y SHA-512. El más seguro es SHA-512 (algoritmo de 80 pasos, el doble tamaño de palabra y tamaño interno que SHA-1). Cuanto más seguro, más lento, se debe encontrar equilibrio entre seguridad y velocidad
  - SHA-3 (NIST 2015). Es lento si se implementa en software, pero más rápido que cualquiera de los estandares anteriores si es implementado en hardware
- Efecto avalancha: El cambio de un solo bit de mensaje afecta a muchos bits del resumen.

- ***Java*** y ***.NET*** proponen una aproximación para implementar la seguridad con ***Security Providers***

```
SHA1 sha = new SHA1CryptoServiceProvider();
// Es una implementacion de la clase abstracta SHA1.
result = sha.ComputeHash(data);
```

- Phyton + Security library: Scapy, hashlib.py,…
- PHP+ Security
- Python y Ruby son los lenguajes de scripting preferidos en la comunidad de seguridad. Python tiene más bibliotecas para implementar la seguridad.

Tecnología **Java para seguridad**



- *Java Cryptography Architecture (JCE)*
  - Para cifrado, generación y acuerdo de claves, códigos de autenticación de mensajes
  - Contiene primitivas criptográficas
  - Se puede extender con paquetes ***Cryptographic Service Provider (CSP).*** Las primitivas que se pueden usar son las implementaciones proporcionadas por todos los *CSPs*
- *Java Secure Sockets Extension (JSSE)*
  - Construir aplicaciones Java con SSL
- *Java Authentication and Authorization Service (JAAS)*
  - Limitar acceso a recursos basado en su identidad

Implementación de la Integridad en Java

- **Security Providers**
  - La clase Provider encapsula el concepto de proveedor de seguridad en Java. Especifica el nombre del provider y la lista de servicios de seguridad que implementa.
  - Multiples providers pueden configurarse al mismo tiempo, y son listados en orden de preferencia. Cuando un servicio de seguridad se requiere, el más alto provider que implementa el servicio es el seleccionado.
  - Ejemplo,
    - *MessageDigest md = MessageDigest.getInstance("MD5");*
  - En el programa se puede indicar también la solicitud de un determinado Provider, indicando su nombre:
    - *MessageDigest md = MessageDigest.getInstance("MD5", "ProvC");*

Implementación de la Integridad en Java

- Resúmenes de **cadenas de caracteres** (paquete java.security.*)
  - Clase ***MessageDigest*** que sirve de base para los resúmenes de mensajes
    - El constructor está protegido, los objetos se crean con el método ***getInstance()***
      - *md= MessageDigest.getInstance("SHA-512");*
    - Procesar los bytes correspondientes
      - *md.update(Cadena.getBytes());*
    - Recuperar el valor del resumen
      - *byte [] md.digest()*
      - **Esto devuelve una matriz de bytes que hay que transformar a *String* si lo queremos visualizar.**
      - *String resumen = Conversor.byteArrayToHexString(byteData);*

Implementación de la Integridad en Python

- Python 2.7.5: Lib/hashlib.py
  - Esta biblioteca implementa diferentes ***secure hash y algoritmos de message digest***: SHA1(160 bits), SHA224, SHA256, SHA384, and SHA512 (definido en FIPS 180-4 Marzo 2012) así como el algoritmo RSA's MD5 (128bits) definido en Internet **RFC 1321**.

```
>>> import hashlib
>>> m=hashlib.sha224()
>>> m.update("probando")
>>> m.digest()
>>> m.hexdigest()
'8e549bd29cd396aa1c712cfc12814df993ee9c30292a8f06eec5d639'
```

Más simplificado

```
>>>hashlib.sha224("probando").hexdigest()
```

Implementación de la Integridad en OpenSSL

- Con el constructor génerico *new()* que toma el nombre de una cadena, representando el algoritmo deseado, permite acceder a los anteriores algoritmos asi como a otros de la biblioteca **OpenSSL**.
- **Los constructores con sus nombres son mucho más rápidos** que el *new()* y deberían usarse preferentemente.
- Uso de new() con un algoritmo proporcionado por OpenSSL:

```
>>> h = hashlib.new('ripemd160')
>>> h.update(“probando")
>>> h.hexdigest()
75b811926a726b50e6073a138c273e0f883fd088'
```

Implementación de la Integridad en Storage.
Integridad en ficheros

- **Monitorización de la Integridad de ficheros** (File integrity monitoring) es un aspecto muy importante de una *Política de seguridad IT proactiva*, que audita y alcanza el cumplimiento de las regulaciones establecidas (tales como PCI DSS or SOX), evitando la alteración de **datos(**Tampering o Data Diddling**).**
  - **Ficheros del sistema**
  - **Ficheros de Password**
  - **Ficheros de Configuración**
  - **Web Pages, cookies, ficheros de formularios**

- Los primeros checksum usados eran un 32 bit CRC (Cyclic Redundancy Check).
  - Los atacantes han demostrado habilidad para modificar un fichero de manera que el CRC checksum podría no detectarlo,...

Secure Hash

- **Imagen ISO:** Archivo que almacena una copia o imagen exacta de un sistema de ficheros,
  - Se rige por el estándar ISO 9660 que le da nombre.
  - Los usos más comunes incluyen la distribución de sistemas operativos, tales como sistemas GNU/Linux, BSD o Live CDs.
  - Se puede usar la función
    - **sha1sum** <nombre del archivo ISO>
    - Solo quedaría comprobar que el hash calculado, coincide con el hash publicado en la web oficial de la distribución GNU/Linux en cuestión.

Implementación de la Integridad en Storage.
Comandos del SO

- En Linux
  - Para hacer el cálculo de la función hash mediante esta herramienta, simplemente debes teclear el siguiente comando:
    - $ **md5sum** <*nombre del archivo*>

- Calculadores Web de message digest online
  - http://www.tools4noobs.com/online_tools/hash/
  - http://hash.online-convert.com/es/generador-sha1 (incluye el cálculo de ficheros)

- **Resúmenes de un flujo de una secuencia de bytes** que se leen o escriben en flujos
  - *Las clases* ***DigestInputStream y DigestOutputStream***
    - Estas clases amplian *java.io.Filter... q*ue los convierten en filtros calculadores de resúmenes.
  - Resúmenes de flujos de entrada

    MessageDigest md=MessageDigest.getInstance("SHA256");
    - *fichero* debe ser un flujo de entrada en Java ***(FileInputStream)***

      DigestInputStream in = new DigestInputStream(fichero, md);
    - Se debe hacer el *read* del flujo ***DigestInputStream***

      byte [] b= new byte[in.available()]

      in.read(b,0,in.available())
    - Recuperar el valor del resumen

      byte[] digest = md.digest();

      String resumen = Conversor.byteArrayToHexString(mdbytes);

Implementación de la Integridad en Storage.
Respuestas a las cuestiones

- Problemas
  - **Colisiones.** Puede modificarse el mensaje sin que cambie el Hash generado
  - **Falsos positivos en la verificación de la integridad**, uno cambia el fichero de datos pero no el hashing values
  - Velocidad de procesamiento/verificación

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- Un técnico se ha hecho cargo de una aplicación **web que almacena 300,000 usernames/passwords en texto plano en un SQL Server**.
- Según el RD 1720/2007 que desarrolla la LOPD para ficheros de datos personales indica en el artículo 93.3
  - El responsable del fichero o tratamiento deberá adoptar las medidas que garanticen la correcta identificación y autenticación de los usuarios.
  - Cuando el mecanismo de autenticación se base en la existencia de contraseñas existirá un procedimiento de asignación, distribución y almacenamiento que **garantice su confidencialidad e integridad.**

Cómo se implementaría??

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- **Problema:** Almacenar los passwords de usuarios como texto en claro puede resultar una vulnerabilidad.
- ¿Solución?: Codificar las contraseñas usando algún algoritmo elaborado en casa que puede agregar un poco de oscuridad, pero no una verdadera "seguridad“
- **Solución:** Cifrar todas las contraseñas en su base de datos utilizando alguna función criptográfica estándar del sector, como Message-Digest Algorithm 5 (MD5).

```
function addNewUser($username, $password){
  global $connection;
  $password = md5($password);
  $q = "INSERT INTO ".TBL_USERS." VALUES ('$username', '$password')";
  return mysql_query($q, $connection);
}
```

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- <?php

```
function checkUserPass($username, $password){
  global $connection;

  $username = str_replace("'","''",$username)
  $password = md5($password);

  // Verifica que el usuario está en la base de datos
   $q = "SELECT password FROM ".TBL_USERS." WHERE username = '$username'";
   $result = mysql_query($q, $connection);
   if(!$result || (mysql_numrows($result) < 1)){
     return 1; //Indicates username failure
   }

   // Retrieve password from result
   $dbarray = mysql_fetch_array($result);

   // Validat si el password es correcto
   if($password == $dbarray['password']){
     return 0; //Success! Username and password confirmed
   }
   else{
     return 1; //Indica fallo del password
   }
 }
```

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- En la autenticación de usuarios: el password presentado por un usuario se le aplica una función hash y se le compara con el hash almacenado.
- Formas de atacar:
  - Tablas arco iris. Previamente a la búsqueda se almacenan en una base de datos los resúmenes de ciertas combinaciones de caracteres.
  - Diccionarios o bases de datos de palabras usuales. Se basa en probar las claves que normalmente se utilizan
  - Fuerza bruta. Se prueban combinaciones de caracteres hasta encontrar la combinación que da un resumen igual al buscado. Para limitar el tiempo se suele reducir los tipos de caracteres a probar.

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

123456 es la contraseña más usada del mundo. Listado de contraseñas usuarios de Adobe

#	Count	Ciphertext	Plaintext
1.	1911938	EQ7fIpT7i/Q=	123456
2.	446162	j9p+HwtWWT86aMjgZFLzYg==	123456789
3.	345834	L8qbAD3jl3jioxG6CatHBw==	password
4.	211659	BB4e6X+b2xLioxG6CatHBw==	adobe123
5.	201580	j9p+HwtWWT/ioxG6CatHBw==	12345678
6.	130832	5djv7ZCI2ws=	qwerty
7.	124253	dQi0asWPYvQ=	1234567
8.	113884	7LqYzKVeq8I=	111111
9.	83411	PMDTbP0LZxu03SwrFUvYGA==	photoshop
10.	82694	e6MPXQ5G6a8=	123123
11.	76910	j9p+HwtWWT8/HeZN+3oiCQ==	1234567890
12.	76186	diQ+ie23vAA=	000000
13.	70791	kCcUSCmonEA=	abc123
14.	61453	ukxzEcXU6Pw=	1234
15.	56744	5wEAInH22i4=	adobe1
16.	54651	WqflwJFYW3+PszVFZo1Ggg==	macromedia
17.	48850	hjAYsdUA4+k=	azerty
18.	47142	rpkvF+oZzQvioxG6CatHBw==	iloveyou
19.	44281	xz6PIeGzr6g=	aaaaaa
20.	43670	Ypsmk6AXQTk=	654321
21.	43497	4V+mGczxDEA=	12345
22.	37407	yp2KLbBiQXs=	666666
23.	35325	2dJY5hIJ4FHioxG6CatHBw==	sunshine
24.	34963	1McuJ/7v9nE=	123321
25.	33452	yxzNxPIsFno=	letmein
26.	32549	dCgB24yq9Bw=	monkey
27.	31554	dA8D8OYD55E=	asdfgh
28.	28349	L8qbAD3jl3jSPm/keox4fA==	password1
29.	28303	zk8NJgAOqc4=	shadow
30.	28132	Ttgs5+ZAZM7ioxG6CatHBw==	princess
31.	27853	pTkQrKZ/JYM=	dragon

Implementación de la Confidencialidad.
Password Integrity and Confidentiality

- https://www.guidingtech.com/61991/cracking-windows-10-password-prevent/

Free XP Rainbow tables

These tables can be used to crack Windows XP passwords (LM hashes). They CANNOT crack Windows Vista and 7 passwords (NT hashes).



Is Your Windows password already cracked?

length:	8	9	10	11	12	13	14	15
LM-enabled	completely cracked	completely cracked	completely cracked	completely cracked	completely cracked	completely cracked	completely cracked	safe
simple passwords	completely cracked	lower, upper, numeric cracked	numeric cracked	numeric cracked	numeric cracked	safe	safe	safe
complex passwords	completely cracked	CLN cracked	CLN cracked	CLN cracked	safe	safe	safe	safe

All data is based on rainbow tables available at FreeRainbowTables.com. Rainbow tables can be used to "decrypt" a password.
"LM-enabled" means the computer has LM password storage turned on. This is the default for Windows 2003/Vista and earlier.
"Complex" means it contains characters from at least three of the four categories (uppercase, lowercase, numbers, symbols).
"CLN" means it is in capital-lowercase-number format (ex. Password123).
Data compiled by Nick Brown (http://nick-brown.com). Last updated: February, 2012.